ADSO Instructora Isaura Suarez

▼ 8/10/2026

▼ Temas vistos:

▼ Algoritmo

Es un conjunto de pasos/instrucciones ordenados y definidos que se siguen para poder resolver un problema o para realizar una tarea

En otras palabras, es una serie de instrucciones que nos indican qué hacer y en qué orden para conseguir un resultado.

▼ Ejemplo de vida cotidiana

preparar un vaso de limonada requiere seguir ciertos pasos para obtener el resultado esperado.

1. Conseguir un vaso, agua, limón y azúcar.
2. Exprimir el limón.
3. Agregar el agua.
4. Añadir azúcar al gusto.
5. Mezclar los ingredientes.
6. Servir la limonada.

Estos pasos forman un algoritmo porque tienen un orden y permiten alcanzar un resultado: preparar limonada.

▼ ¿Para qué sirven los algoritmos?

Los algoritmos sirven para:

- Resolver problemas: encontrar una solución siguiendo pasos definidos.
- Organizar tareas: establecer qué se debe hacer primero y qué después.

- Automatizar procesos: permitir que una computadora realice tareas sin que una persona deba indicar cada paso manualmente.
- Desarrollar programas: establecer las instrucciones que debe seguir un software.
- Evitar errores: definir un procedimiento claro para obtener el resultado esperado.

▼ Características

Un algoritmo debe tener las siguientes características:

Característica	Explicación	Ejemplo
Preciso	Sus instrucciones deben ser claras.	Escribir «agregar 2 cucharadas de azúcar» en lugar de «agregar un poco».
Ordenado	Los pasos deben seguir una secuencia lógica.	Primero abrir el programa y después utilizarlo.
Finito	Debe terminar después de una cantidad limitada de pasos.	Completar una receta y obtener el plato preparado.
Definido	Con las mismas condiciones iniciales, debe producir el mismo resultado previsto.	Sumar 2 + 3 siempre da 5.
Tiene entradas y salidas	Puede recibir datos y producir un resultado.	Recibir dos números y mostrar su suma.

Importante: no todos los algoritmos necesitan recibir datos de una persona. Algunos trabajan con información obtenida automáticamente.

▼ ¿Cómo funciona un algoritmo?

Un algoritmo puede entenderse mediante tres elementos principales:
Entrada

Los datos que recibe el algoritmo.
Ejemplo: 5 y 3

Proceso
Las operaciones que realiza.
Ejemplo: sumar 5 + 3

Salida
El resultado obtenido.
Ejemplo: 8

Este modelo se conoce como Entrada → Proceso → Salida (EPS).
No todos los algoritmos necesitan una entrada externa, pero este modelo es muy útil para comprender cómo se resuelven muchos problemas informáticos.

▼ Ejemplo en programación

Imagina que necesitas crear un programa que calcule la suma de dos números.

El algoritmo sería:

1. Solicitar el primer número.
2. Solicitar el segundo número.
3. Sumar los dos números.
4. Mostrar el resultado.

En pseudocódigo, que es una forma de escribir las instrucciones sin depender de un lenguaje de programación específico, quedaría así:

```
INICIO
    Leer numero1
    Leer numero2
    resultado = numero1 + numero2
    Escribir resultado
FIN
```

Si ingresamos los números 10 y 5, el programa mostraría 15.

En este caso:

- Entrada: 10 y 5.
- Proceso: sumar los números.
- Salida: 15.

▼ ¿Cuál es la diferencia entre un algoritmo y un programa?

Aunque están relacionados, no son lo mismo.

Algoritmo	Programa
Es una secuencia lógica de pasos para resolver un problema.	Es un conjunto de instrucciones escritas en un lenguaje de programación que una computadora puede ejecutar.
Puede expresarse con palabras, diagramas o pseudocódigo.	Se escribe en lenguajes como Python, Java o JavaScript.
Describe cómo solucionar el problema.	Permite ejecutar esa solución en una computadora.

Ejemplo: el algoritmo indica que se deben sumar dos números. El programa contiene el código necesario para realizar esa suma.

Un algoritmo puede existir sin convertirse en un programa, pero para que una computadora ejecute una solución, sus instrucciones deben representarse de una forma que pueda procesar.

▼ Formas de representar un algoritmo

Existen diferentes maneras de representar un algoritmo:

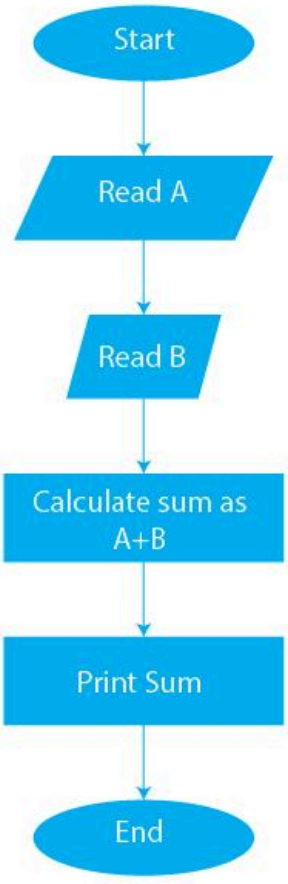
- Lenguaje natural: se explica mediante palabras comunes.
- Pseudocódigo: se escriben instrucciones estructuradas parecidas a las de un programa, pero sin seguir obligatoriamente la sintaxis de un lenguaje específico.
- Diagrama de flujo: se utilizan figuras y flechas para representar los pasos y las decisiones.
- Código de programación: se implementan las instrucciones mediante un lenguaje como Python o JavaScript.

▼ Tipos de estructuras que utilizan los algoritmos

En programación, los algoritmos suelen combinar tres estructuras fundamentales:

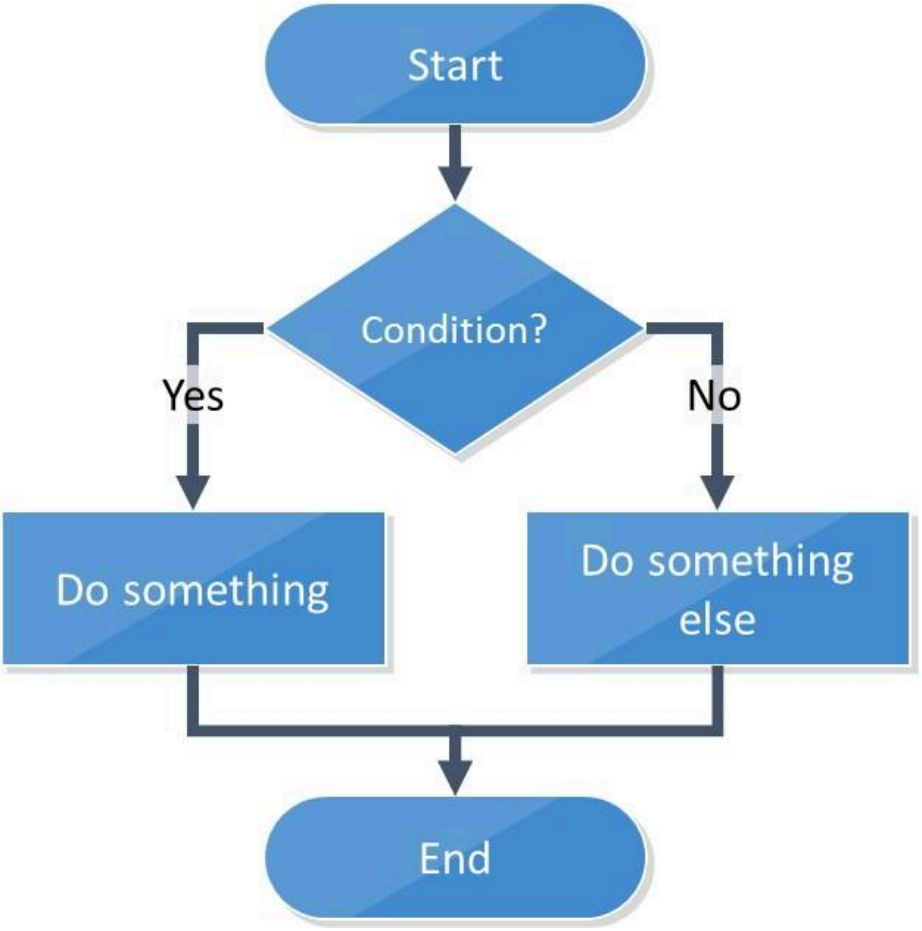
▼ Secuencial

Las instrucciones se ejecutan una después de otra, en un orden establecido.
Ejemplo: leer dos números, sumarlos y mostrar el resultado.



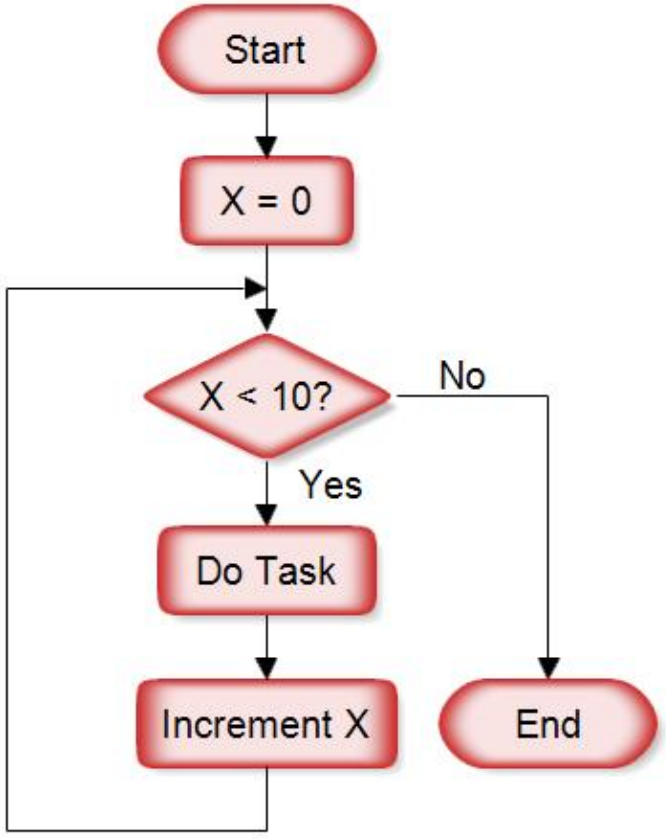
▼ Condicional o selectiva

Permite tomar decisiones dependiendo de si una condición se cumple o no.
Ejemplo: si una persona tiene 18 años o más, mostrar «Mayor de edad»; de lo contrario, mostrar «Menor de edad».



▼ Repetitiva o iterativa

Permite repetir una o varias instrucciones mientras se cumpla una condición o durante un número determinado de veces.
Ejemplo: mostrar los números del 1 al 10.



▼ Conceptos clave para recordar

- Algoritmo: pasos ordenados para resolver un problema.
- Entrada: datos que recibe un algoritmo.
- Proceso: operaciones que se realizan con los datos.
- Salida: resultado producido.
- Pseudocódigo: representación de un algoritmo mediante instrucciones estructuradas.
- Diagrama de flujo: representación gráfica de los pasos de un algoritmo.
- Programa: instrucciones implementadas en un lenguaje de programación para que una computadora las ejecute.
- Depuración: proceso de identificar y corregir errores en un programa.

En resumen: un algoritmo es la lógica de una solución. Antes de escribir código, puedes diseñar el algoritmo para tener claro

qué debe hacer el programa y cómo debe hacerlo.

▼ Variable

Una variable es un espacio en la memoria de la computadora donde se almacena un dato que puede utilizarse durante la ejecución de un programa y cuyo valor puede cambiar.
En palabras sencillas, una variable es como una caja que tiene un nombre y guarda información.

▼ ¿Para qué sirven?

Las variables permiten:

- Almacenar información.
- Realizar operaciones matemáticas.
- Guardar resultados de cálculos.
- Utilizar datos dentro de un programa.
- Actualizar información cuando sea necesario.

Ejemplo: en un programa de una tienda, podemos guardar el precio de un producto, la cantidad disponible y el nombre del cliente mediante variables.

▼ ¿Cómo se crea una variable?

Observa este ejemplo en Python:

```
edad = 18nombre = "Marian"altura = 1.65
```

Cada línea crea una variable y le asigna un valor.

Variable	Valor	Tipo de dato
`edad`	`18`	Entero (`int`)
`nombre`	`"Marian"`	Cadena de texto (`str`)
`altura`	`1.65`	Decimal (`float`)

El símbolo `=` se utiliza para asignar un valor a una variable. No significa exactamente lo mismo que la igualdad matemática.

Por ejemplo:

```
edad = 18edad = 19
```

Primero, la variable `edad` guarda el valor `18`. Después, su valor cambia a `19`.

▼ Tipos de datos que puede almacenar una variable

▼ Enteros (`int`)

Números sin parte decimal.

Ejemplos: `5`, `18`, `-3`.

▼ Decimales (`float`)

Números que tienen parte decimal.

Ejemplos: `3.14`, `1.65`, `7.5`.

▼ Cadenas de texto (`str`)

Almacenan palabras, frases o caracteres.

Ejemplos: `"Hola"`, `"Marian"`, `"Bogotá"`.

▼ Booleanos (`bool`)

Representan dos valores lógicos: verdadero o falso.

Ejemplos: `True`, `False`.

Nota: estos son tipos de datos comunes en Python. Otros lenguajes pueden manejarlos de forma diferente.

▼ Reglas para nombrar variables

Al crear una variable, debemos seguir ciertas reglas. En Python:

- El nombre puede contener letras, números y guiones bajos (`_`).
- No puede comenzar con un número.
- No debe contener espacios.
- No puede utilizarse una palabra reservada del lenguaje.

- Se recomienda elegir nombres que expliquen qué información almacenan.

Nombre	¿Es válido?
`edad`	Sí
`nombre_usuario`	Sí
`numero2`	Sí
`2numero`	No
`nombre usuario`	No
`class`	No, es una palabra reservada en Python

▼ Ejemplo práctico con variables

Imagina que quieres calcular el precio total de tres cuadernos que cuestan $4.000 cada uno.

```
precio = 4000cantidad = 3total = precio * cantida
dprint(total)
```

Resultado en pantalla:

```
12000
```

¿Qué sucede?

1. `precio` guarda el valor `4000`.
2. `cantidad` guarda el valor `3`.
3. `total` almacena el resultado de multiplicar `precio` por `cantidad`.
4. `print(total)` muestra el resultado en pantalla.

▼ Diferencia entre variable y constante

- Variable: su valor puede cambiar durante la ejecución del programa.
- Constante: representa un valor que se pretende mantener sin cambios.

Ejemplo en Python:

```
edad = 18edad = 19PI = 3.14159
```

En este caso, `edad` cambia de `18` a `19`. El nombre `PI`, escrito en mayúsculas por convención, representa un valor que se espera mantener constante. Python no impide modificarlo, pero se recomienda no hacerlo.

▼ Resumen para estudiar

Una variable es un espacio de memoria identificado por un nombre que permite almacenar y utilizar datos en un programa. Su valor puede cambiar según las instrucciones ejecutadas. Las variables pueden guardar distintos tipos de datos, como números enteros, decimales, textos y valores booleanos.
Truco para recordarlo: piensa en una variable como una caja etiquetada. La etiqueta es el nombre, lo que hay dentro es el valor y puedes cambiar su contenido cuando el programa lo necesite.

▼ Variables contadoras y sumadoras

▼ Variable contadora

Una variable contadora es una variable que se utiliza para contar cuántas veces ocurre algo o cuántas veces se ejecuta una instrucción.

Generalmente, aumenta o disminuye en una cantidad fija, muchas veces de uno en uno.

Ejemplo cotidiano: contar cuántos estudiantes ingresan a un salón.

```
contador = 0contador = contador + 1contador =
contador + 1contador = contador + 1print(conta
dor)
```

Resultado:

```
3
```

¿Qué pasó?

1. El contador comienza en `0`.
2. Llega el primer estudiante y aumenta a `1`.
3. Llega el segundo y aumenta a `2`.
4. Llega el tercero y aumenta a `3`.

La variable `contador` permite saber cuántos estudiantes han ingresado.

▼ Variable sumadora o acumuladora

Una variable sumadora es una variable que almacena y acumula el resultado de sumar diferentes valores a medida que se ejecuta un algoritmo.

A diferencia de la contadora, que normalmente aumenta en una cantidad fija, la sumadora agrega valores que pueden ser diferentes.

Ejemplo cotidiano: calcular el total de las compras de una tienda.

```
sumadora = 0sumadora = sumadora + 2000sumadora
= sumadora + 3500sumadora = sumadora + 1500pri
nt(sumadora)
```

Resultado:

```
7000
```

¿Qué pasó?

1. La sumadora comienza en `0`.
2. Se agregan `$2.000`: el total es `$2.000`.
3. Se agregan `$3.500`: el total es `$5.500`.
4. Se agregan `$1.500`: el total es `$7.000`.

La variable `sumadora` guarda el total acumulado.

▼ Diferencias entre contadora y sumadora

Característica	Contadora	Sumadora
Función	Contar eventos o repeticiones.	Acumular valores.
Qué agrega	Normalmente una cantidad fija, como `1`.	Valores que pueden ser diferentes.
Valor inicial habitual	`0`	`0`
Ejemplo	Contar 10 estudiantes.	Sumar las notas de 10 estudiantes.
Operación típica	`contador = contador + 1`	`suma = suma + nota`

▼ ¿Cómo se utilizan dentro de un ciclo?

Imagina que quieres sumar las notas de tres estudiantes y contar cuántas notas ingresaste.

```
contador = 0sumadora = 0for nota in [4, 3, 5]:
contador = contador + 1    sumadora = sumadora
+ notaprint("Cantidad de notas:", contador)pri
nt("Suma de notas:", sumadora)
```

Resultado:

```
Cantidad de notas: 3
Suma de notas: 12
```

En este algoritmo se utilizan las dos variables al mismo tiempo:

- `contador` cuenta cuántas notas se procesaron.
- `sumadora` acumula el valor de las notas.

Si después quisieras calcular el promedio, podrías dividir la suma entre la cantidad de notas: \(12 \div 3 = 4\).

▼ Truco para diferenciarlas

- Contadora: ¿cuántos hay?

- Sumadora: ¿cuánto suman en total?

Una precisión importante para tus apuntes: las variables contadoras y sumadoras no son los únicos dos tipos de variables que existen. Son dos usos específicos de las variables dentro de un algoritmo. También existen variables para almacenar nombres, números, resultados, estados y muchos otros datos.

▼ Tipos de datos

Los tipos de datos son categorías que indican qué clase de información puede almacenar una variable y qué operaciones se pueden realizar con ella.

En palabras sencillas, cuando creamos una variable, debemos saber qué tipo de información queremos guardar: un número, un texto, un valor verdadero o falso, entre otros.

Por ejemplo, no es lo mismo guardar la edad de una persona que guardar su nombre.

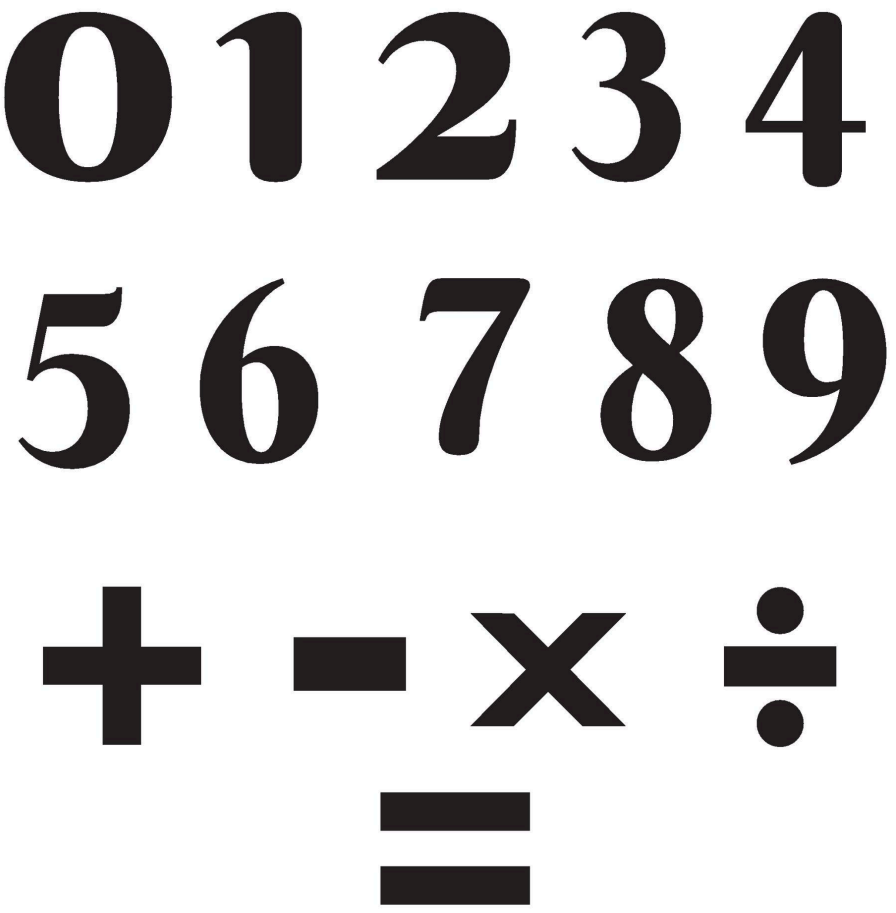
```
edad ← 18
nombre ← "Marian"
estatura ← 1.65
estudia ← VERDADERO
```

▼ Entero (Integer)

Representa números sin parte decimal. Puede incluir números positivos, negativos y el cero.

Ejemplos: `10`, `-5`, `0`, `200`.

Se utiliza para almacenar edades en años cumplidos, cantidades de productos o el número de estudiantes.

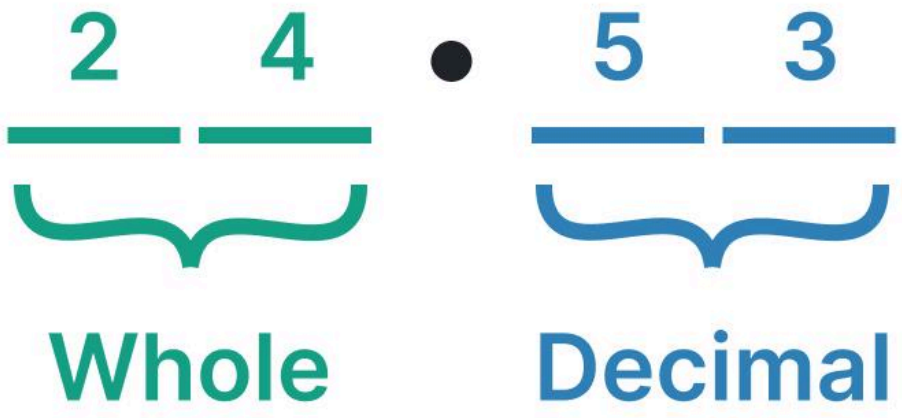


▼ Real o decimal (Float)

Representa números que pueden tener una parte decimal.

Ejemplos: `3.14`, `2.5`, `-7.8`.

Se utiliza para representar estaturas, temperaturas, promedios o medidas.

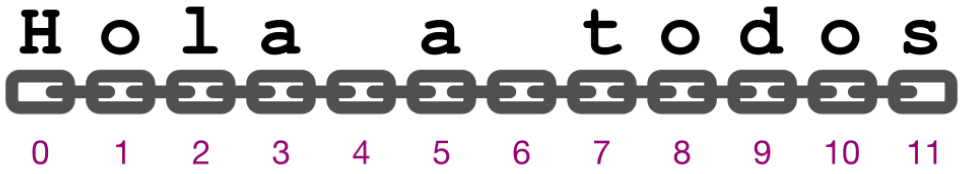


▼ Cadena de caracteres (String)

Representa texto formado por letras, números, espacios o símbolos.

Ejemplos: `"Hola"`, `"Marian"`, `"Calle 10"`, `"12345"`.

Se utiliza para nombres, direcciones, mensajes y códigos que deben tratarse como texto.

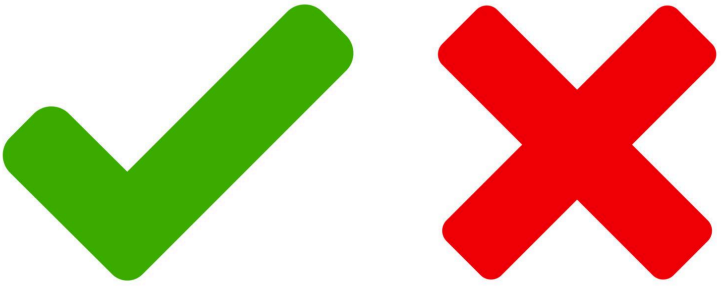


▼ Booleano o lógico (Boolean)

Representa únicamente dos valores lógicos: verdadero o falso.

Ejemplos: `VERDADERO`, `FALSO`.

Se utiliza para comprobar condiciones, como saber si un usuario está registrado o si una nota es suficiente para aprobar.

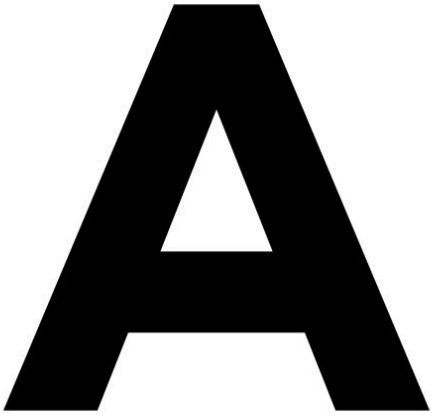


▼ Carácter (Char)

Representa un solo carácter, como una letra, un dígito o un símbolo.

Ejemplos: `'A'`, `'7'`, `'#'`.

Se utiliza cuando solo se necesita almacenar un carácter individual.



Importante: los tipos exactos y sus nombres dependen del lenguaje de programación. Por ejemplo, Python no tiene un tipo `char` independiente: un solo carácter se representa como una cadena de texto.

▼ ¿Cómo se utilizan los tipos de datos en pseudocódigo?

En pseudocódigo, podemos declarar variables para indicar qué información vamos a utilizar.

Por ejemplo:

```
INICIO
    ENTERO edad
    REAL promedio
    CADENA nombre
    LOGICO aprobado

    edad ← 18
    promedio ← 4.5
```

```
    nombre ← "Marian"
    aprobado ← VERDADERO

    ESCRIBIR edad
    ESCRIBIR promedio
    ESCRIBIR nombre
    ESCRIBIR aprobado
FIN
```

En este ejemplo:

Variable	Tipo de dato	Valor
`edad`	Entero	`18`
`promedio`	Real	`4.5`
`nombre`	Cadena	`"Marian"`
`aprobado`	Lógico	`VERDADERO`

La forma de declarar variables puede cambiar según las convenciones del pseudocódigo que utilice tu profesor.

▼ Diferencia entre un número y un texto

Esta diferencia es fundamental para programar.

Valor	Tipo de dato	¿Se puede sumar numéricamente?
`25`	Entero	Sí
`25.5`	Real	Sí
`"25"`	Cadena de texto	No directamente
`"Hola"`	Cadena de texto	No numéricamente
`VERDADERO`	Lógico	No como operación aritmética habitual

Observa la diferencia:

```
numero ← 5
resultado ← numero + 3
```

Resultado: `8`.

En cambio:

```
texto ← "5"
```

Aquí se guarda el carácter `5` como texto, no como número. Para realizar operaciones matemáticas con él, normalmente hay que convertirlo a un tipo numérico.

▼ ¿Cómo elegir el tipo de dato correcto?

Piensa en qué información necesitas almacenar:

- ¿Una cantidad de estudiantes? Entero.
- ¿El promedio de un estudiante? Real.
- ¿El nombre de una persona? Cadena.
- ¿Si un usuario inició sesión? Lógico.
- ¿Una sola letra como categoría? Carácter, si el lenguaje lo admite.

Un detalle importante: un número de identificación, un teléfono o un código postal normalmente conviene almacenarlo como texto, aunque contenga números, porque no se utiliza para hacer operaciones matemáticas y puede tener ceros al inicio.

▼ Operadores

Los operadores son símbolos o palabras que permiten realizar operaciones con datos y variables.

▼ Operadores aritméticos

Se utilizan para realizar operaciones matemáticas.

Operador	Nombre	Ejemplo	Resultado
`+`	Suma	`5 + 3`	8
`-`	Resta	`5 - 3`	2
`*`	Multiplicación	`5 * 3`	15
`/`	División	`6 / 2`	3
`%`	Módulo o residuo	`7 % 2`	1
`**`	Potencia en Python	`2 ** 3`	8

Importante: el operador módulo `%` devuelve el residuo de una división. Por ejemplo, `7 % 2` es igual a `1` porque sobra uno.

▼ Operadores relacionales o de comparación

Se utilizan para comparar dos valores. El resultado de la comparación es verdadero o falso.

Operador	Significado	Ejemplo	Resultado
`>`	Mayor que	`8 > 5`	Verdadero
`<`	Menor que	`3 < 7`	Verdadero
`>=`	Mayor o igual que	`5 >= 5`	Verdadero
`<=`	Menor o igual que	`4 <= 3`	Falso
`==`	Igual a	`5 == 5`	Verdadero
`!=`	Diferente de	`5 != 3`	Verdadero

No confundas estos dos operadores:

- `=` suele representar una asignación: guardar un valor en una variable.
- `==` representa una comparación: comprobar si dos valores son iguales.

Por ejemplo:

```
edad = 18print(edad == 18)
```

El resultado es `True`, porque la edad almacenada es igual a 18.

▼ Operadores lógicos

Permiten combinar condiciones o invertir su resultado.

Operador	Significado	Ejemplo	Resultado
`AND` / `and`	Y	`5 > 2 AND 8 > 4`	Verdadero
`OR` / `or`	O	`5 < 2 OR 8 > 4`	Verdadero
`NOT` / `not`	Negación	`NOT (5 > 2)`	Falso

En Python se escriben `and`, `or` y `not`. En pseudocódigo se pueden escribir `Y`, `O` y `NO`, según la convención utilizada.

- Y: ambas condiciones deben ser verdaderas.
- O: al menos una condición debe ser verdadera.
- NO: invierte el valor lógico.

Ejemplo: para ingresar a una actividad, una persona debe tener al menos 18 años y presentar su documento. Ambas condiciones deben cumplirse.

▼ 14/10/2026

▼ Condicionales

Las condicionales son estructuras de control que permiten que un algoritmo tome decisiones según se cumpla o no una condición.
En palabras sencillas, sirven para indicarle a un programa qué debe hacer en diferentes situaciones.
Por ejemplo, si una persona obtiene una nota igual o superior a 3, aprueba. Si obtiene una nota inferior a 3, reprueba.
El programa evalúa la condición y decide qué instrucciones ejecutar.

▼ ¿Cómo funcionan las condicionales?

Una condición es una expresión que puede ser verdadera (VERDADERO) o falsa (FALSO).

Condición

¿La nota es mayor o igual a 3?

Verdadero

Mostrar «Aprobado»

Falso

Mostrar «Reprobado»

Las condicionales suelen utilizar operadores relacionales como `>`, `<`, `>=`, `<=`, `==` y `!=`.

▼ Tipos de estructuras condicionales

▼ Condicional simple: SI

Se utiliza cuando queremos ejecutar una instrucción solamente si se cumple una condición.

Ejemplo: mostrar una alerta cuando la temperatura supere los 35 grados.

```
INICIO
    LEER temperatura

    SI temperatura > 35 ENTONCES
        ESCRIBIR "Alerta: Temperatura alta"
    FIN SI
FIN
```

Si la temperatura es 38, aparece la alerta. Si es 30, no se muestra nada.

Clave: si la condición es falsa, el programa simplemente omite las instrucciones del `SI`.

▼ Condicional doble: SI - SINO

Se utiliza cuando tenemos dos alternativas: una si la condición es verdadera y otra si es falsa.

Ejemplo: determinar si una persona es mayor o menor de edad.

```
INICIO
    LEER edad

    SI edad >= 18 ENTONCES
        ESCRIBIR "Mayor de edad"
    SINO
        ESCRIBIR "Menor de edad"
    FIN SI
FIN
```

Si la edad es 20, muestra «Mayor de edad». Si es 16, muestra «Menor de edad».

Clave: se ejecuta una alternativa u otra, nunca ambas en la misma evaluación de esta estructura.

▼ Condicional múltiple: SI - SINO SI - SINO

Se utiliza cuando necesitamos evaluar varias condiciones diferentes.

Ejemplo: clasificar una nota.

```
INICIO
    LEER nota

    SI nota >= 4.5 ENTONCES
        ESCRIBIR "Excelente"
    SINO SI nota >= 3 ENTONCES
        ESCRIBIR "Aprobado"
    SINO
        ESCRIBIR "Reprobado"
    FIN SI
FIN
```

¿Cómo funciona?

- Nota de 4.8: muestra «Excelente».
- Nota de 3.5: muestra «Aprobado».
- Nota de 2.0: muestra «Reprobado».

Las condiciones se evalúan en orden. Cuando una se cumple, se ejecuta su bloque y se omiten las alternativas restantes.

En algunos lenguajes se utiliza `SINO SI`; en otros, como Python, se escribe `elif`.

▼ Condicionales anidadas

Son condicionales ubicadas dentro de otras condicionales.

Se utilizan cuando una decisión depende de otra decisión previa.

Ejemplo: comprobar si una persona tiene la edad requerida y, después, si tiene una entrada.

```
INICIO
    LEER edad
    LEER tieneEntrada

    SI edad >= 18 ENTONCES
        SI tieneEntrada == VERDADERO ENTONCES
            ESCRIBIR "Puede ingresar"
        SINO
            ESCRIBIR "Necesita una entrada"
        FIN SI
    SINO
        ESCRIBIR "No cumple la edad requerida"
    FIN SI
FIN
```

Primero se comprueba la edad. Solo si cumple ese requisito se evalúa la entrada.

▼ Operadores que se utilizan en las condicionales

Operador	Significado	Ejemplo
`>`	Mayor que	`edad > 18`
`<`	Menor que	`edad < 18`
`>=`	Mayor o igual que	`nota >= 3`
`<=`	Menor o igual que	`nota <= 5`
`==`	Igual a	`edad == 18`
`!=`	Diferente de	`edad != 18`
`Y` / `AND`	Ambas condiciones verdaderas	`edad >= 18 Y tieneEntrada`
`O` / `OR`	Al menos una condición verdadera	`esEstudiante O esEmpleado`
`NO` / `NOT`	Invierte el valor lógico	`NO esEstudiante`

Recuerda que `=` se utiliza habitualmente para asignar un valor, mientras que `==` sirve para comparar dos valores en lenguajes como Python.

▼ Ejemplo de clase

```
Si (condición) entonces
  Instruciones
  Instruciones
  Instrucciones
FinSi
```

```
Si (condicion) entonces
   Instrucciones
   Instrucciones
   Instrucciones
SiNo
   Instrucciones
   Instrucciones
```

```
   Instrucciones
FinSi
```

```
Anidado:

Si (condicion) entonces
   Instrucciones
   Instrucciones
   Instrucciones
SiNo
   Si (condicion) entonces
     Instrucciones
     Instrucciones
     Instrucciones
   SiNo
     Instrucciones
     Instrucciones
     Instrucciones
    FinSi
 Finsi
```

▼ Ejemplo completo con variables y operadores

Problema: determinar si una persona puede aprobar una asignatura, considerando que necesita una nota mínima de 3.

```
INICIO
    REAL nota

    LEER nota

    SI nota >= 3 ENTONCES
        ESCRIBIR "Aprobaste la asignatura"
    SINO
        ESCRIBIR "No aprobaste la asignatura"
    FIN SI
FIN
```

Aquí aparecen varios conceptos:

- Variable: `nota`.

- Tipo de dato: `REAL`.
- Entrada: `LEER nota`.
- Operador relacional: `>=`.
- Condicional: `SI...ENTONCES...SINO`.
- Salida: `ESCRIBIR`.

▼ Errores comunes

- Confundir `=` con `==`.
- Olvidar que `>=` incluye la igualdad.
- Pensar que una condición falsa siempre produce un error. No es así: simplemente puede ejecutar otra alternativa o no hacer nada.
- No respetar el orden de las condiciones múltiples.
- Confundir `Y` con `O`: con `Y` deben cumplirse todas las condiciones; con `O` basta con que se cumpla al menos una.
- Olvidar cerrar las estructuras según la convención de pseudocódigo utilizada.

▼ 15/10/2026